

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5117

Name: Challa Priyadarshini

Reg no. : 192311118

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-based

Question 1:

Secure Data Transmission using Block Cipher

A. Why does the issue occur in ECB mode?

Electronic Codebook (ECB) mode encrypts each plaintext block **independently** using the same encryption key.

- If two plaintext blocks are **identical**, ECB produces **identical ciphertext blocks**.
- As a result, repeated patterns in the plaintext remain visible in the ciphertext.
- An attacker can identify these patterns and gain information about the original data, even without knowing the encryption key.
- This makes ECB **unsuitable for transmitting sensitive or repetitive data**, such as financial records.

Example:

- Plaintext: Salary = \$5000 | Salary = \$5000
- ECB Ciphertext: X1 | X1 (same ciphertext block repeated)

B. Suggest a more secure block cipher mode and justify your choice.

A better choice is **Cipher Block Chaining (CBC) mode**.

Justification:

- CBC uses an **Initialization Vector (IV)** and XORs each plaintext block with the previous ciphertext block before encryption.
- Even if two plaintext blocks are identical, their ciphertext blocks will be **different** because each block depends on the previous encrypted block.
- This removes visible patterns from the ciphertext and provides much better confidentiality than ECB.

Advantages of CBC:

- Hides repeating patterns in plaintext.
- More secure for transmitting financial and confidential data.
- Widely used for secure block encryption (when implemented with a random IV and proper integrity protection).

Conclusion

- **A.** ECB reveals patterns because identical plaintext blocks produce identical ciphertext blocks.
- **B.** Use **CBC** (or, preferably in modern applications, **AES-GCM**) because it eliminates repeated ciphertext patterns and offers much stronger security for financial data transmission

Question 2:

Choosing an Encryption Algorithm for Banking System

A. Most suitable algorithm

The most suitable algorithm is 3-DES (Triple DES) among the given options.

B. Justification

- DES uses a 56-bit key, which is too small and can be broken using modern brute-force attacks. Therefore, it is not suitable for banking systems.
- 3-DES applies DES encryption three times, providing a much stronger effective security level than DES.
- Blowfish is also a strong symmetric encryption algorithm and is faster than 3-DES in many software implementations.
- However, for the given choices, 3-DES provides stronger security than DES and was historically used in banking and financial systems.
- The main disadvantage of 3-DES is that it is slower than Blowfish because encryption is performed three times.
- Therefore, when high security is the main priority and moderate performance is acceptable, 3-DES is the appropriate choice.

Conclusion

3-DES is the best choice because it provides significantly better security than DES while maintaining acceptable performance for a banking application.

Note: In modern banking systems, **AES** is generally preferred over DES, 3-DES, and Blowfish because it provides stronger security and better efficiency.

Question 3:

Key Exchange in an Unsecured Network

A. How Diffie-Hellman Key Exchange Works

Diffie-Hellman (DH) allows two users to create a **shared secret key** over an unsecured network without directly sending the secret key.

Steps:

1. Both users agree on two public values:
 - Prime number **p**
 - Generator **g**
2. **User A** chooses a private key **a**.
3. **User B** chooses a private key **b**.
4. User A calculates:
 $A = g^a \bmod p$
and sends **A** to User B.
5. User B calculates:
 $B = g^b \bmod p$
and sends **B** to User A.
6. User A calculates the shared secret:
 $K = B^a \bmod p$
7. User B calculates:
 $K = A^b \bmod p$
8. Both obtain the same shared key:
 $K = g^{ab} \bmod p$

The shared key can then be used for symmetric encryption of their communication.

B. Security Threat and Solution

Threat: Man-in-the-Middle (MITM) Attack

- Diffie-Hellman by itself does **not authenticate** the users.
- An attacker can position themselves between User A and User B.

- The attacker can establish separate keys with each user and potentially interfere with their communication.

Solution: Authentication

Use Digital Signatures or Digital Certificates along with Diffie-Hellman.

- Digital signatures verify the identity of the communicating users.
- Certificates can be issued by a trusted **Certificate Authority (CA)**.
- This prevents an attacker from successfully impersonating either user.

Conclusion

Diffie-Hellman provides secure key establishment, but authentication must be added to prevent MITM attacks. A combination such as Authenticated Diffie-Hellman + digital certificates/signatures is much safer for communication over an unsecured network.

Question 4:

Public Key Cryptography for Email Security

A. RSA Encryption and Decryption

RSA uses two keys:

- **Public Key** → shared with others.
- **Private Key** → kept secret by the owner.

For secure email communication:

1. The sender obtains the receiver's **public key**.
2. The sender encrypts the message using the receiver's public key.
3. The encrypted message is sent through the network.
4. The receiver uses their **private key** to decrypt the message.
5. Only the receiver with the correct private key can decrypt the message.

Basic process:

Plaintext → RSA Encryption + Receiver's Public Key → Ciphertext → RSA Decryption + Receiver's Private Key → Plaintext

RSA can also be used for **digital signatures**, where the sender signs using their private key and the receiver verifies using the sender's public key.

B. Key Distribution and Management

Key management is important because RSA security depends on protecting and correctly distributing keys.

- **Public keys** can be distributed openly.
- **Private keys** must be kept secret and protected from unauthorized access.
- **Digital certificates** can verify that a public key actually belongs to the intended person or organization.
- A **Certificate Authority (CA)** can issue and manage certificates.
- Keys should be **rotated/replaced** when necessary.
- Compromised keys should be **revoked** immediately.

Conclusion:

RSA provides secure email communication, but proper **key distribution, authentication, storage, and protection of private keys** are essential for maintaining security.

Question 5:

Cryptography for IoT Devices

A. Constraints of IoT Devices

IoT devices usually have limited resources compared with computers and servers.

Main constraints include:

- Limited processing power
- Small memory/storage
- Low battery capacity
- Limited bandwidth
- Need for fast communication
- Need for low computational and power consumption
- Devices may operate continuously for long periods.

RSA requires relatively large key sizes and computationally expensive operations, which can consume more CPU time, memory, and energy.

B. Recommended Cryptographic Approach

ECC (Elliptic Curve Cryptography) is the better choice for low-power IoT devices.

Reasons:

- ECC provides strong security with **smaller key sizes**.
- Smaller keys require **less memory**.
- ECC operations can require **less computational power** than equivalent RSA security.

- Lower computation can help reduce **battery consumption**.
- Smaller keys also reduce the amount of data transmitted over the network.
- It is well suited to **resource-constrained IoT devices**.

For example, a commonly cited comparison is:

RSA 1024-bit $\approx$ **ECC 160-bit** in approximate security strength.

Conclusion

ECC is more suitable than RSA for low-power IoT devices because it provides strong security while requiring smaller keys, less memory, lower communication overhead, and generally lower computational resources.